

PROMPT PLAYBOOK

Building Changelog Master with AI-Assisted Development

- Real prompts from a production build
 - Refined versions for better results
- Reusable templates for any project
 - 8 core prompting principles

By Mark Kashef | Prompt Advisers

Contents

Phase 1	Initial Build	3
Phase 2	Multi-Source Support	5
Phase 3	Design Overhaul	7
Phase 4	UX Refinements	9
Phase 5	Chat & Polish	11
Principles	8 Prompting Rules	13
Templates	Reusable Patterns	16

Phase 1: Initial Build

Original Prompt

"Build me a changelog tracking app that fetches Claude Code's changelog from GitHub, analyzes it with Gemini, generates audio, and sends email notifications."

Refined Prompt

Build a full-stack changelog monitoring application:

CORE FUNCTIONALITY:

1. Fetch changelog from GitHub raw URL
2. Parse markdown into structured version objects
3. Categorize items as: feature, fix, removal, breaking
4. Analyze with Gemini 3 Flash API - return JSON
5. Generate audio using Gemini 2.5 Flash TTS
6. Send email notifications via Resend API

TECH STACK:

- Frontend: React 19 + TypeScript + Vite
- Backend: Express.js + better-sqlite3
- Styling: Tailwind CSS 4

CACHING:

- Cache AI analysis by version hash in SQLite
- Cache TTS audio by text hash in SQLite

UI:

- Two tabs: "Changelog" and "What Matters"
- Audio player with speed control, voice selection
- Dark/light mode toggle

✓ Why This Works Better

- Specific API models named

- Data structures described
- Caching strategy included
- Implementation order given

Phase 2: Multi-Source Support

Original Prompt

"Allow me to monitor multiple changelog sources, not just Claude Code."

Refined Prompt

```
Add multi-source changelog monitoring:

DATABASE:
changelog_sources table:
- id TEXT PRIMARY KEY
- name TEXT NOT NULL
- url TEXT NOT NULL UNIQUE
- is_active INTEGER DEFAULT 1
- last_version TEXT
- last_checked_at TEXT

API ENDPOINTS:
- GET /api/sources - List all sources
- POST /api/sources - Create { name, url }
- PATCH /api/sources/:id - Update source
- DELETE /api/sources/:id - Delete source

FRONTEND:
- SourcesPanel component with add/edit/delete
- URL test button before saving
- Source dropdown in header
- Persist selection in localStorage
```

✓ Why This Works Better

- Full database schema provided
- Complete API contract
- UI component named

- State persistence specified

Phase 3: Design Overhaul

Original Prompt

"Make this look more like Anthropic's design."

Refined Prompt

Redesign with Anthropic-inspired design system:

COLORS (Tailwind CSS 4 @theme):

```
/* Light backgrounds */  
cream-50: #faf8f5  
cream-100: #f5f1eb
```

```
/* Dark backgrounds */  
charcoal-800: #191f2a  
charcoal-900: #0f1419
```

```
/* Primary accent */  
coral-500: #ff6b5b
```

```
/* Secondary accent */  
teal-500: #14b8a6
```

DESIGN RULES:

- cream-50 light background, charcoal-900 dark
- Coral for primary actions
- Teal for success states
- rounded-xl cards, rounded-full badges

UPDATE ALL COMPONENTS with new colors.
Keep functionality identical.

✓ Why This Works Better

- Exact hex values provided
- No color interpretation needed
- Consistent across all components

Phase 4: UX Refinements

Original Prompt

"Make the favicon more vibrant / Only expand most recent release / Allow dark mode as default setting"

Refined Prompt

Three UX improvements:

1. FAVICON:

- Bold colors for small sizes (16x16, 32x32)
- Vibrant orange (#ff6b35) background
- White document icon with teal badge

2. CHANGELOG VIEW:

- Only expand most recent version by default
- Change: `versions.slice(0, 3)` -> `versions.slice(0, 1)`

3. DEFAULT THEME:

- Add "Appearance" section to `SettingsPanel`
- Toggle: "Use dark mode by default"
- `useTheme` hook: add `setDefaultTheme()`
- Store in `localStorage` as 'defaultTheme'
- Init priority: manual > default > system

✓ Why This Works Better

- Multiple related changes batched
- Specific code changes noted
- Reduces context switching

Phase 5: Chat & Polish

Original Prompt

"Remove sentiment analysis / Action items should be dashes / Add history / Better formatted TLDR / Chat should never use em dashes"

Refined Prompt

Improve What Matters and Chat:

MATTERS VIEW:

1. Remove sentiment badge completely
2. Action items: checkboxes to dash bullets
3. Analysis history:
 - Fetch from GET /api/analysis on mount
 - Dropdown: "2.0.74 (current)", "2.0.73"
 - "Viewing archived" banner when browsing
 - "Back to current" button
4. TL;DR: wrap in ReactMarkdown

CHAT SYSTEM PROMPT:

Add rule:

"NEVER use em dashes (–) or en dashes (—).
Use regular hyphens (-) or colons (:) instead."

✓ Why This Works Better

- Related changes grouped
- Specific component code mentioned
- System prompt rule explicitly stated

Prompting Principles

1. Be Specific About Data Structures

■ Weak:

"Store the changelog data"

■ Strong:

Store as ChangelogVersion[]: { version: string, date: string, items: ChangelogItem[] }

2. Name Your APIs

■ Weak:

"Use Google's AI"

■ Strong:

"Use Gemini 3 Flash (gemini-3-flash-preview) for analysis with JSON mode"

3. Describe Caching Strategy

■ Weak:

"Cache the results"

■ Strong:

"Cache AI analysis in SQLite by version hash. Check cache before API call."

4. Specify UI Behavior

■ Weak:

"Add a settings panel"

■ Strong:

"SettingsPanel as modal from gear icon. Sections: Appearance, Auto-refresh, Notifications."

5. Give Color Codes

■ Weak:

"Make it look better"

■ Strong:

"Use cream-50 (#faf8f5) for light bg, coral-500 (#ff6b5b) for accents."

6. Batch Related Changes

■ Weak:

3 separate prompts for related changes

■ Strong:

"Three changes to MattersView: (1) Remove X, (2) Change Y, (3) Add Z"

7. Include What NOT to Do

■ Weak:

"Generate audio"

■ Strong:

"Generate audio using TTS. NEVER call API if audio exists in cache."

8. Specify System Prompts

■ Weak:

"Make chat responses cleaner"

■ Strong:

"Add to system prompt: NEVER use em dashes. Use hyphens instead."

Reusable Templates

New Feature Template

```
Add [FEATURE NAME]:

PURPOSE: [One sentence]

DATABASE (if needed):
[Table schema]

API ENDPOINTS:
- [METHOD] [PATH] - [Description]

FRONTEND:
- Component: [Name]
- Location: [Where in UI]
- State: [What it manages]

BEHAVIOR:
1. [Step 1]
2. [Step 2]

EDGE CASES:
- [What if X]
```

Design Change Template

```
Update design for [AREA]:

COLORS:
- Primary: [hex]
- Secondary: [hex]

SPACING:
- Padding: [values]
- Gap: [values]

COMPONENTS TO UPDATE:
- [Component1.tsx]
```

- [Component2.tsx]

KEEP UNCHANGED:

- [What not to modify]

Bug Fix Template

Fix [BUG]:

CURRENT: [what happens]

EXPECTED: [what should happen]

ROOT CAUSE: [why]

FILES:

- [file.ts] - [what's wrong]

FIX:

[How to solve]

Key Takeaways

■ Specific tech choices

Name models, libraries, and versions explicitly

■ Data structures upfront

Define shapes before coding begins

■ Caching strategies

Prevent expensive API calls with smart caching

■ Batched changes

Group related work into single prompts

■ Explicit rules

Say what NOT to do, not just what to do

■ System prompts

Control AI behavior at the source

*These prompts evolved over multiple sessions.
Use them as templates for your own projects.*